

Nesne Yönelimli Analiz ve Tasarım

Nesne Tasarımı

Tasarım Desenleri
Design Patterns

Tasarım Desenleri (Design Patterns)

- * Yazılımlar geliştirilirken karşılaşılan genel tasarım problemlerini tanımlayarak, bu problemin en uygun nasıl çözülebileceğini ((high coherence, low coupling) kod tekrar kullanımını artırmak ve değişikliği kolaylaştırmak için) ve çözümün ortaya çıkaracağı sonuçları anlatır.
- * Yazılımların tasarımıyla ilgili genel problemlerin çözümüne yönelik deneyimlerin aktarımını sağlar.
- * Programlama dillerinden bağımsızdır.
- * Yazılım geliştiricilerin tasarımlarla ilgili tartışma yapmasını kolaylaştırır.
- * Tasarım desenleri dört bölümden oluşur
 - * Desenin adı
 - * problemin tanımı: desenin ne zaman/herede kullanılabileceği
 - * çözüm: problemin nasıl çözüleceği (kullanılması gereken bileşenler, aralarındaki bağıntı vb.)
 - * sonuç: deseni uygulamanın sonuçları?

Tasarım Desenleri (Design Patterns)

- * İlk olarak Erich Gamma vd. tarafından yazılan aşağıdaki kitapla, yazılım geliştirmede tasarım deseni konusu ortaya çıkmıştır. Bu kitapta 23 tasarım deseni yer almaktadır. Bunlar dışında da çok sayıda desen bulunmaktadır.
- * Creational(Nesne oluşturma): Nesnelerin uygun bir şekilde oluşturulmasını sağlayacak mekanizmalar içerir.
- * Structural(Yapısal): Nesnelerin sistemler içerisine uygun olarak yerleştirilmesini sağlayacak desenlerdir.
- * Behavioral(Davranışsal): Nesnelerin birbirleriyle uygun olarak etkileşiminini düzenleyen mekanizmalar.

Creational (Nesne oluşturma)	Structural (Yapısal)	Behavioral (Davranışsal)
Singleton Pattern	Adapter Pattern	Template Method Pattern
Factory Pattern	Composite Pattern	Mediator Pattern
Abstract Factory Pattern	Proxy Pattern	Chain of Responsibility Pattern
Builder Pattern	Flyweight Pattern	Observer Pattern
Prototype Pattern	Facade Pattern	Strategy Pattern
	Bridge Pattern	Command Pattern
	Decorator Pattern	State Pattern
		Visitor Pattern
		Interpreter Pattern
		Iterator Pattern
		Memento Pattern

Tasarım Desenleri : Singleton

- * Amaç: Bir sınıfın yalnızca tek nesnesi olmasını ve bu nesneye global olarak erişilmesini sağlamak.
- * Nesne oluşturmaya (creational) ilgili desenlerden biridir.
- * Nesne yoksa oluşturulur döndürülür, varsa olan döndürülür.
- * Nesnenin new komutu ile dışarıdan oluşturulabilmesini engellemek için yapıcı yöntem "private" yapılır.
- * Yapıcı gibi geçen "static" bir yöntem tanımlanır. Nesne oluşturma görevi bu yöntemindir. Oluşturulan nesne "static" bir üyede saklanır.

Tasarım Desenleri : Singleton

Soru1: Bu modülü, ATM uygulaması içerisine ekleyiniz. ATM sisteminde kart doğrulama, kullanıcı doğrulama ve bakiye değişimi bilgilerinin log dosyasına yazılmasını sağlayınız.

LogYoneticisi	
instance	LogYoneticisi
out	PrintWriter
LogYoneticisi(String)	
getInstance(String)	LogYoneticisi
dosyayaYaz(String)	void

Uygulama	
main(String[])	void



```
public class LogYoneticisi {
```

```
    private static LogYoneticisi instance;  
    PrintWriter out;
```

```
    private LogYoneticisi(String logDosyasi){  
        try {  
            out = new PrintWriter(new FileWriter(logDosyasi,true), true);  
        } catch (IOException e) {e.printStackTrace();}  
    }
```

```
    public static synchronized LogYoneticisi getInstance(String logDosyasi){  
        if(instance==null)  
            instance = new LogYoneticisi(logDosyasi);  
        return instance;  
    }
```

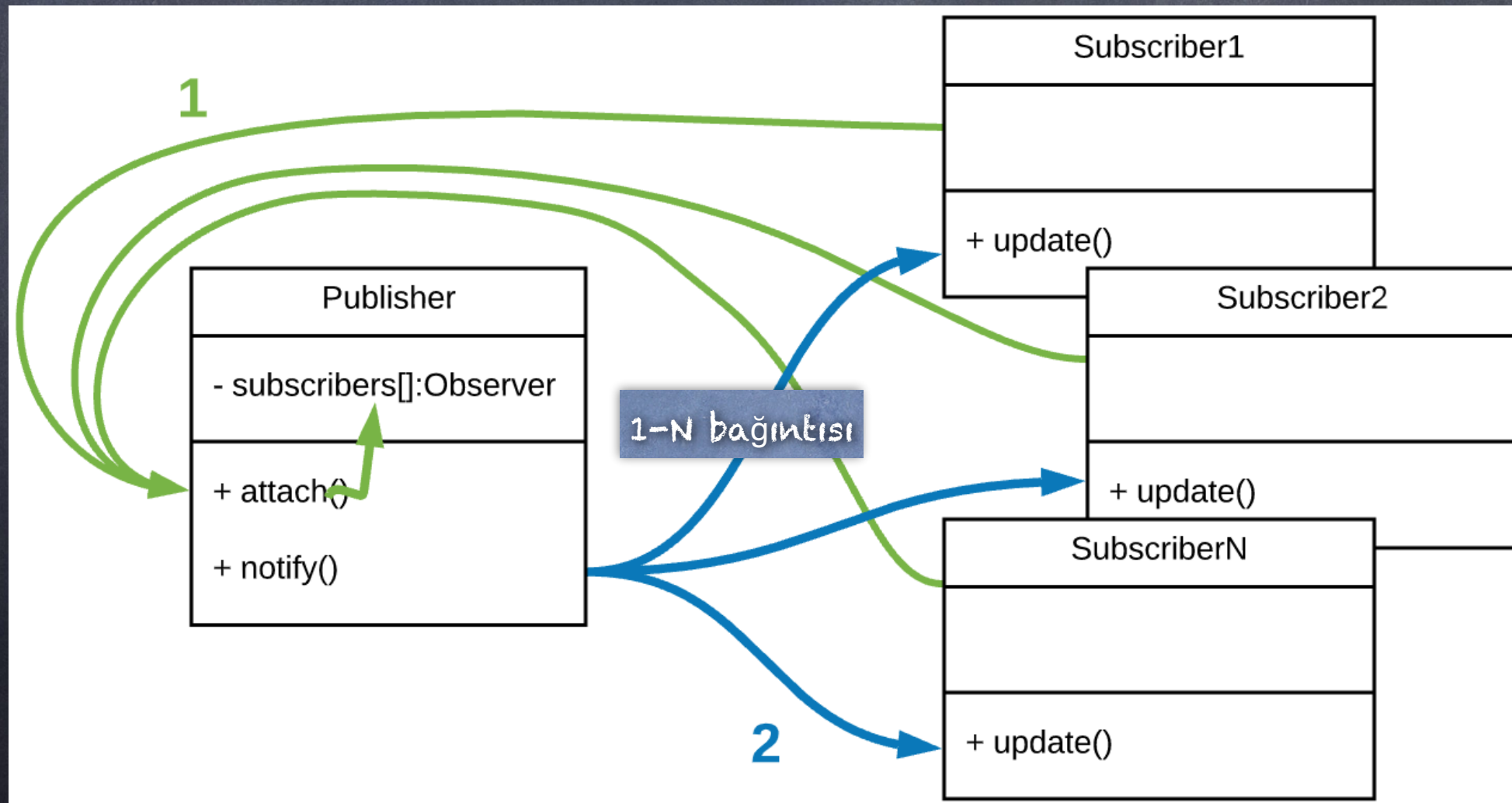
```
    public void dosyayaYaz(String mesaj) {  
        out.println(LocalDate.now()+":"+mesaj);  
    }
```

```
}
```

```
public class Uygulama {  
    public static void main(String [] args){  
        LogYoneticisi.getInstance("Log.txt").dosyayaYaz("[WARNING]:uyari mesajı 1");  
    }  
}
```


Tasarım Desenleri : Observer (Gözlemci)

- * Çok sayıda nesneye, gözlemledikleri nesnede meydana gelen olayı bildirmek gerektiğinde kullanılabilir.
- * Davranışsal desenlerden biridir.
- * Kullanım örnekleri:
 - * mağazaya ürün geldiğinde ilgili müşterilere bildirim gönderilmesi, ürün indirime girdiğinde bildirim gönderilmesi
 - * bakiye değiştiğinde müşteri, loglama sistemi ve gerçek zamanlı görüntüleme/ anormal durum tespit sistemine bilgi gönderilmesi vb.
- * Böyle bir etkileşim Publish(Yayın)-Subscribe(abone) olarak da adlandırılır.



Tasarım Desenleri : Observer (Gözlemci)

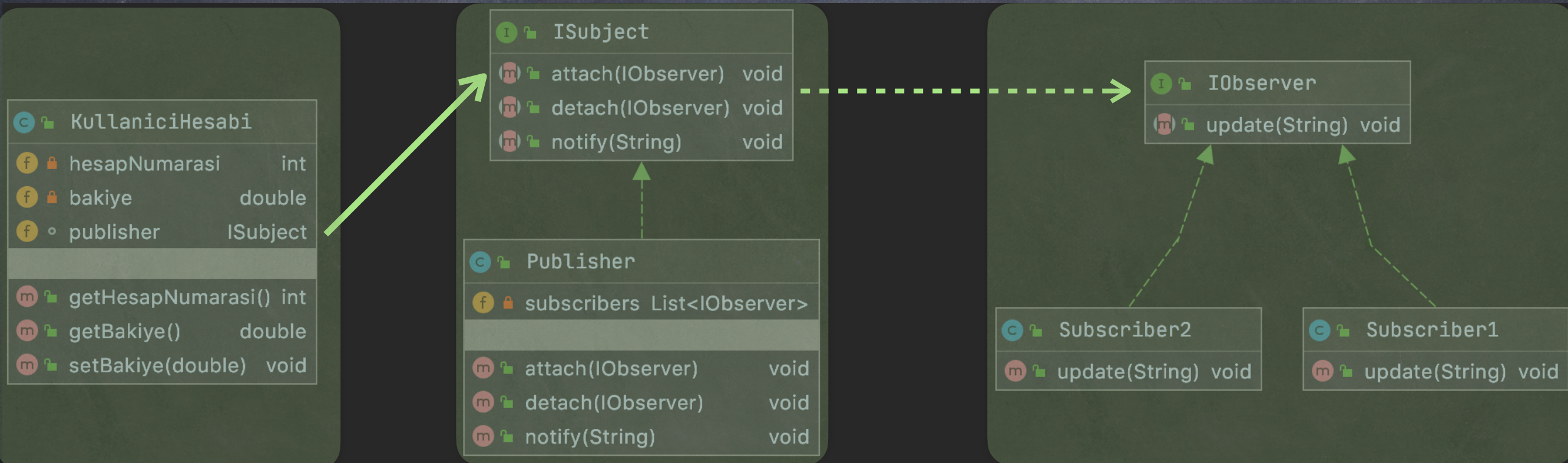
- * Subject (publisher- yayıncı): gözlemcilerin kaydedilmesi/gıkartılması, istemciideki değışikliklerin gözlemcilere bildirilmesi işlemlerinden sorumludur.
- * Observer (subscriber-abone): istemciideki olayların yansıtıldığı/gönderildiği nesne.
- * İstemci (kullaniciHesabi): olayın üretildiği nesne.

Örnek: Kullanıcının bakiyesi değıştiğinde, bu olayı abonelere bildiren uygulama.

istemci(event source)

subject (publisher)

observer (subscriber)



Tasarım Desenleri : Observer (Gözlemci)

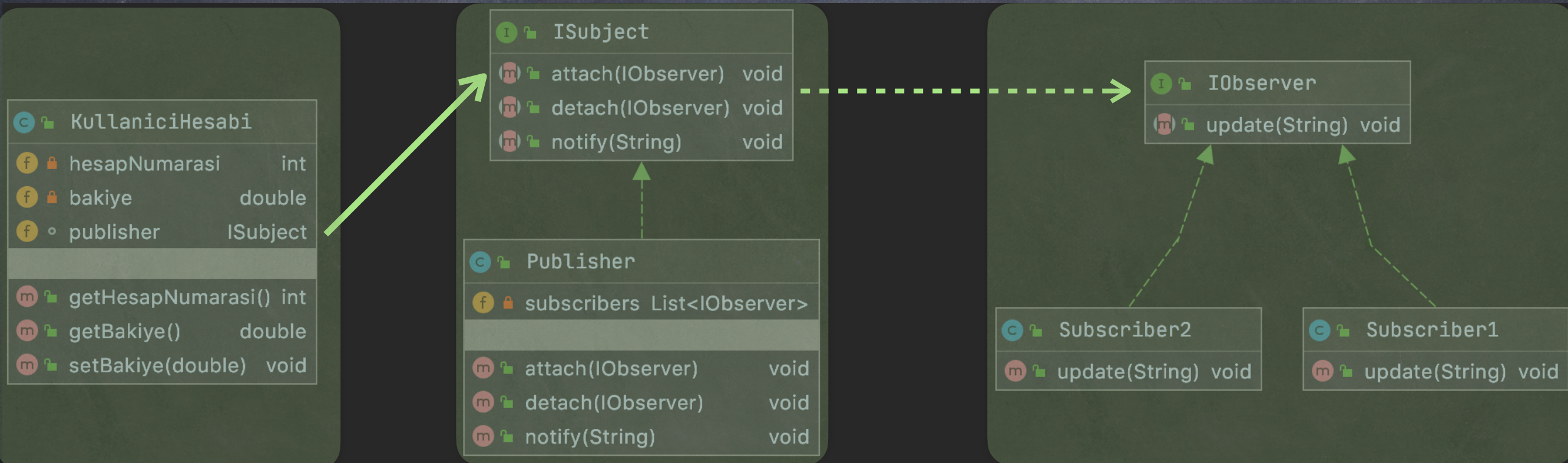
Soru1: Bu modülleri ATM sistemine ekleyiniz. Bakiye değiştiğinde bu değişiklik ilgili abonelere gönderilsin. Abonelerden biri içerisinde LogYoneticisi ile log alma işlemi gerçekleştirilsin.

Örnek: Kullanıcının bakiyesi değiştiğinde, bu olayı abonelere bildiren uygulama.

istemci(event source)

subject (publisher)

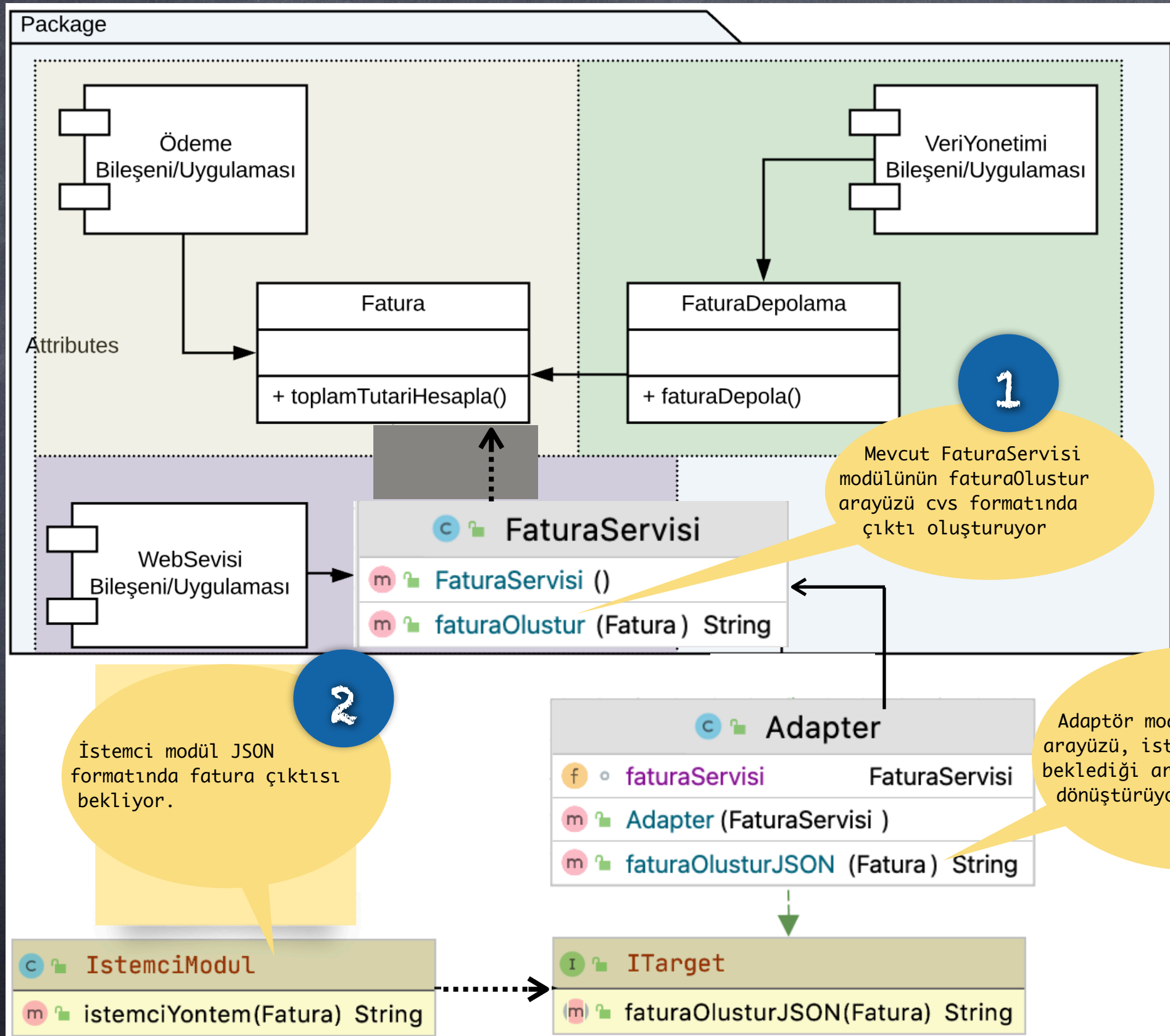
observer (subscriber)



Tasarım Desenleri : Adapter

- * Yapısal desenlerden biridir.
- * Bir sınıfın arayüzünü istemci tarafından beklenen başka bir arayüze dönüştürmek için kullanılır.
- * Mevcut sınıfların (kaynak kodunu değiştirmeden) diğer sınıflarla çalışabilmesini sağlar.
- * Bu desen sayesinde, daha önceden geliştirilmiş modülleri, yeni modüller ile birlikte (bu modüllere uygun arayüzlere sahip olmadığı durumlarda) kullanabiliriz.

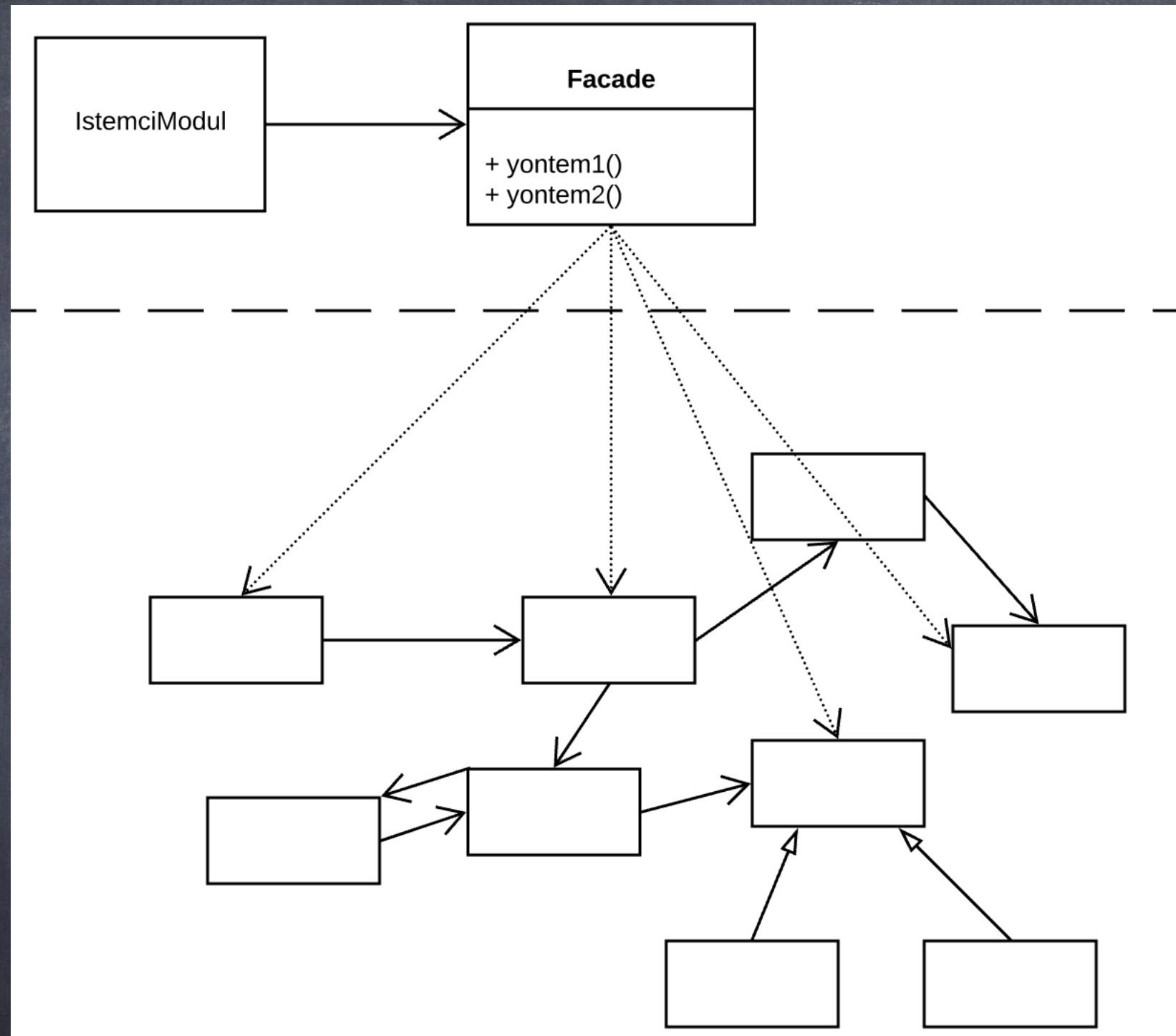
Tasarım Desenleri : Adapter



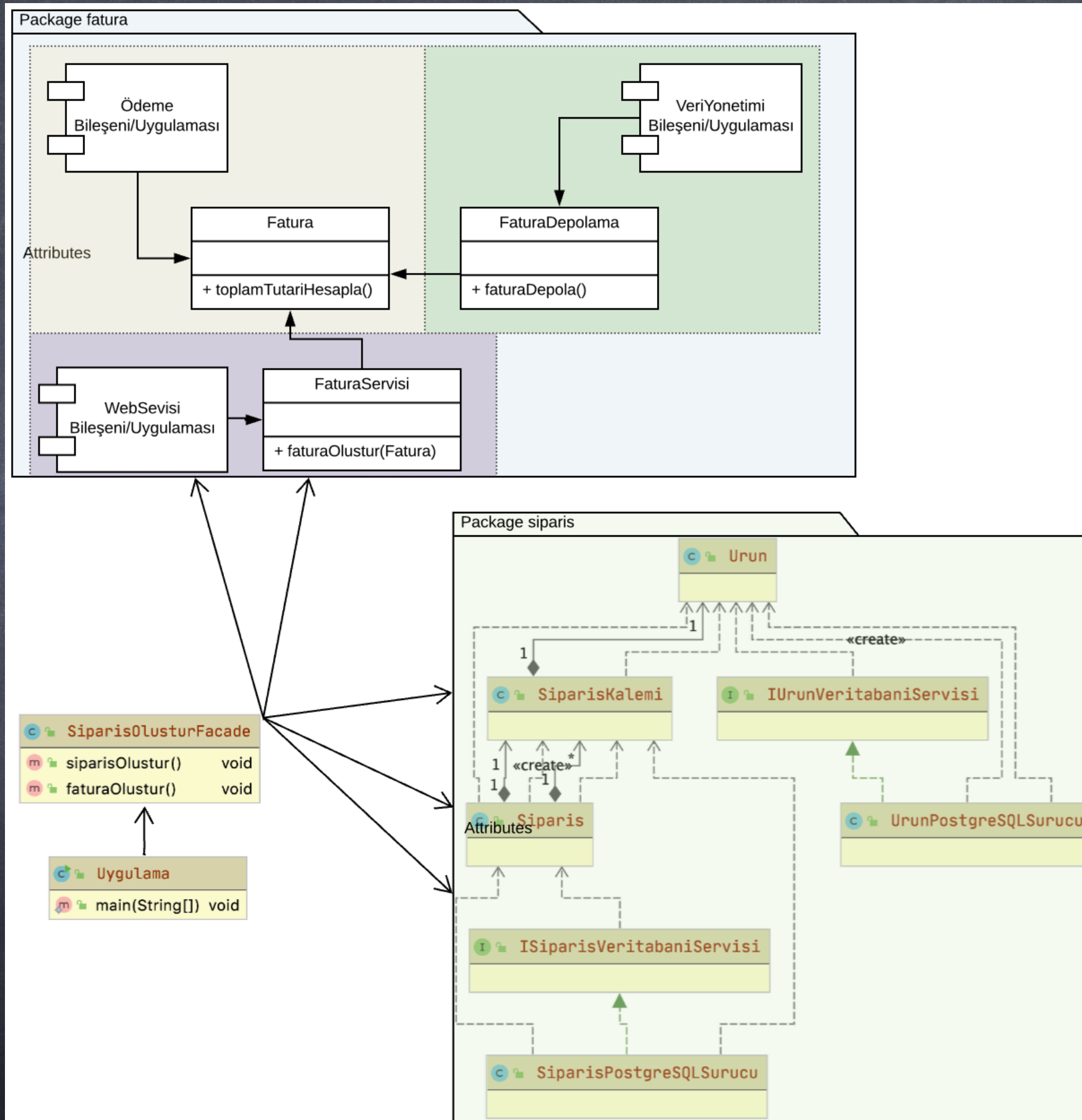
Tasarım Desenleri : Facade

- * Yapısal desenlerden biridir.
- * Karmaşık yapıdaki bir sınıf topluluğu (kütüphane, alt bileşen, eskiden yazılmış kodlar vb.) için basitleştirilmiş arayüz sağlar.
- * İstemci kodun karmaşık alt sistemle etkileşimini kolaylaştırır (loosly coupling).
- * İyileştirilme imkanı olmayan eski modüllerle etkileşim imkanı sağlar.
- * Anlaşılabilirliği artırır.
- * Yazılım içerisinde katmanlar oluşturulmasına imkan verir.

Tasarım Desenleri : Facade



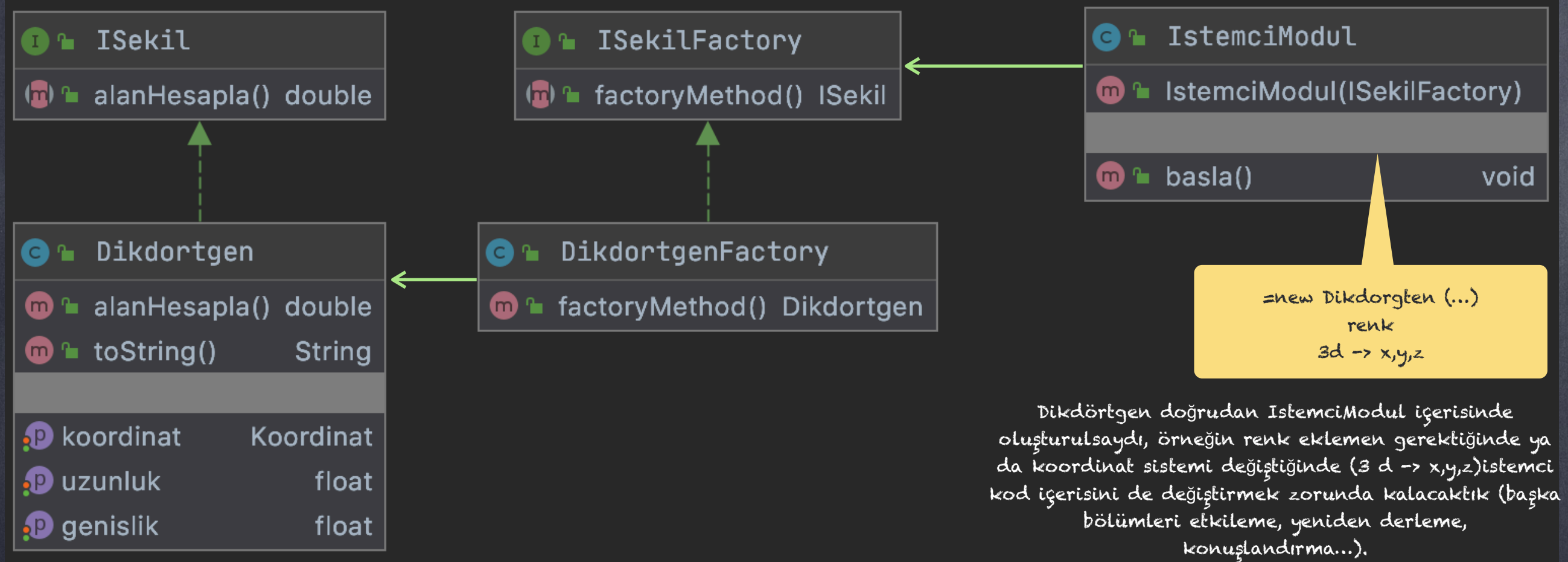
Tasarım Desenleri : Facade



Tasarım Desenleri-Factory Method

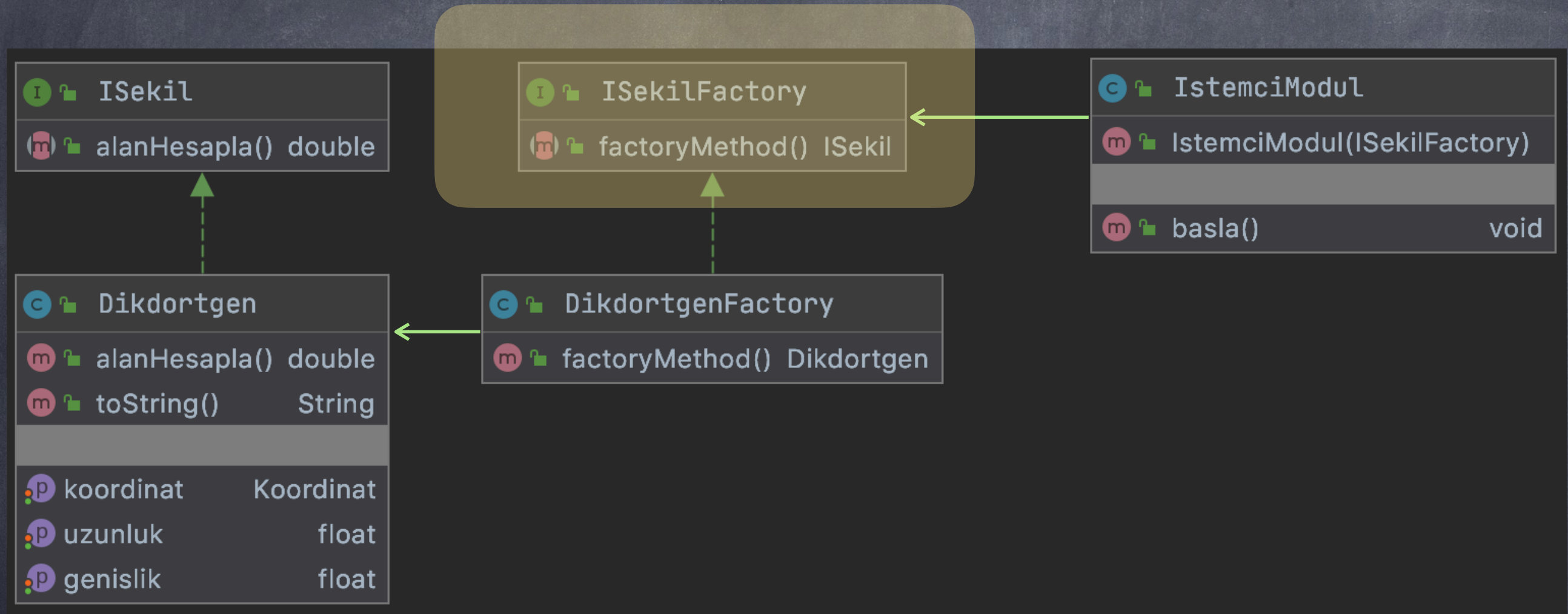
- * Nesne oluşturmaya (creational) ilgili desenlerden biridir.
- * Bir sınıftan nesne oluşturmak gerektiğinde, bu sorumluluğu istemci koddan ayırmak (kapsülleme/SRP) için kullanılır.
- * Özellikle çok sayıda parametre göndermek gerektiğinde (kalıtım söz konusu ise) nesne oluşturma işi karmaşılaşır (kod tekrarı, değişikliklerden istemci kodun etkilenmesi, kodların kötü görünmesi...).
- * Nesne oluşturmak gerektiğinde "factory method" çağırılarak nesne oluşturulur. Böylece; sınıfın bulunduğu yol/paket, istisna yönetimi vb. işlerle uğraşmaya gerek kalmaz.
- * Nesne oluşturmaya ilgili herhangi bir değişiklikten (yolların değişimi, parametre değişimleri vb.), istemci kod etkilenmez (SRP).

Tasarım Desenleri-Factory Method (uygulama1)



Tasarım Desenleri-Factory Method (uygulama1)

1. İstemci modül içerisindeki şekil dikdörtgen olacaksa ve değişme ihtimali yoksa DI uygulamaya gerek yok.



Soru1: ATM uygulamasında Banka Bilgi Sistemi nesnesi oluşturma işlemini "factory method" deseniyle yapınız.

Tasarım Desenleri-Factory Method (uygulama2)

